MERN STACK

TRAINING TR-103 REPORT DAY 18

15 July 2026

Overview:

The eighteenth day of the MERN Stack training focused on building interactive React applications using **useState**, **useEffect**, event handling, and array state management. We developed a **Todo List** application to understand dynamic rendering and state updates. We also explored the **useEffect Hook** for handling side effects and created a **Counter** application to practice state management in React.

Learning Objectives:

- Build a Todo List application using React.
- Understand array state management.
- Learn the `map()` method for rendering lists.
- Understand the `useEffect` Hook and its dependency array.
- Build a Counter application using React Hooks

REACT

Todo List

A Todo List is a task management application that allows users to add and display tasks. It demonstrates how React manages dynamic data using components and state.

Features:

- Add new tasks.
- Display tasks dynamically.
- Uses React state management.
- Updates the UI automatically.

React Component

A React Component is a reusable piece of the user interface that contains both structure and logic. The Todo List is implemented as a functional component.

Features:

- Reusable.
- Easy to maintain.
- Makes applications modular.

useState Hook

The useState Hook is used to create and update state variables in React. It allows components to store data that changes over time.

Features:

- Stores dynamic data.
- Updates the user interface automatically.
- Simplifies state management.

Event Handling

Event handling allows React applications to respond to user actions.

Common Events:

- onChange – Captures user input.
- onClick – Executes an action when clicked.

Features:

- Handles user interactions.
- Updates application state.
- Makes applications interactive.

Array State Management

React stores multiple Todo items in an array. New tasks are added using the spread operator (...), which preserves existing data while adding new items.

Features:

- Stores multiple values.
- Updates state efficiently.
- Prevents loss of existing data.

map() Method

The map() method is used to iterate through an array and display each item dynamically.

Features:

- Loops through array elements.
- Generates UI dynamically.
- Commonly used for rendering lists.

useEffect Hook

The useEffect Hook is used to perform side effects in React functional components. It executes code after a component renders.

Common Uses:

- Fetching data from APIs.
- Setting timers.
- Updating the document title.
- Working with local storage.
- Adding or removing event listeners.

Dependency Array:

- useEffect(() => {}) → Runs after every render.
- useEffect(() => {}, []) → Runs only once.
- useEffect(() => {}, [state]) → Runs when the specified state changes.

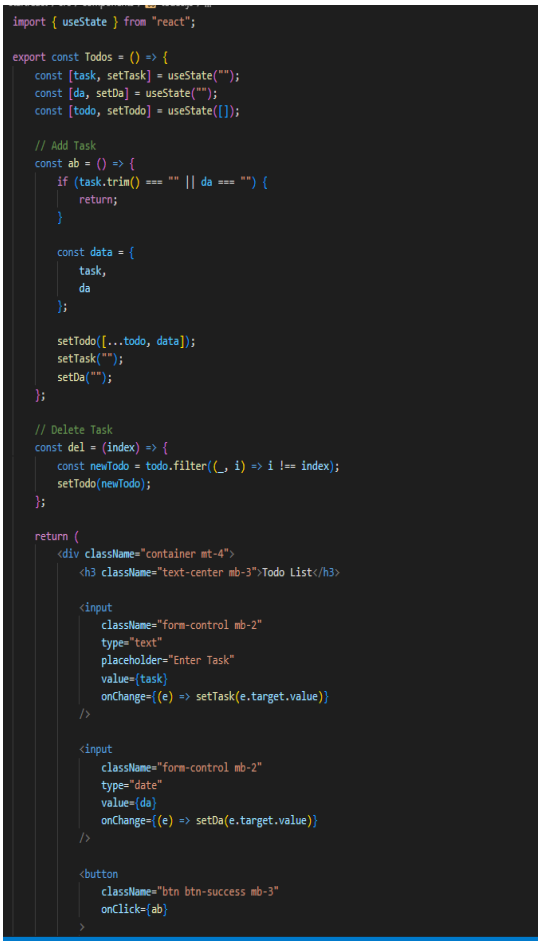
Counter Application

A Counter application was created using the useState Hook to increase or decrease a numerical value based on user interaction.

Features:

- Increment counter.
- Decrement counter.
- Dynamic UI updates.
- Uses React state management.

TASK - 1:



```jsx
import { useState } from "react";

export const Todos = () => {
    const [task, setTask] = useState("");
    const [da, setDa] = useState("");
    const [todo, setTodo] = useState([]);

    // Add Task
    const ab = () => {
        if (task.trim() === "" || da === "") {
            return;
        }

        const data = {
            task,
            da
        };

        setTodo([...todo, data]);
        setTask("");
        setDa("");
    };

    // Delete Task
    const del = (index) => {
        const newTodo = todo.filter((_, i) => i !== index);
        setTodo(newTodo);
    };

    return (
        <div className="container mt-4">
            <h3 className="text-center mb-3">Todo List</h3>

            <input
                className="form-control mb-2"
                type="text"
                placeholder="Enter Task"
                value={task}
                onChange={(e) => setTask(e.target.value)}
            />

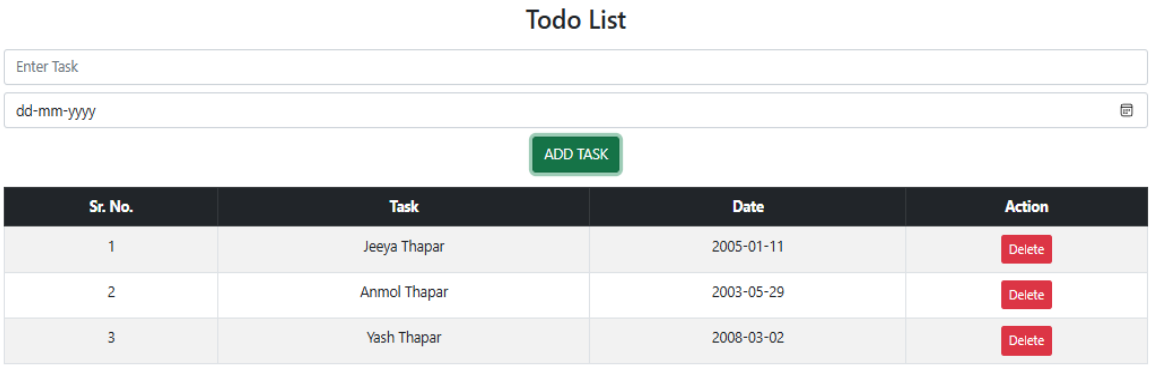
            <input
                className="form-control mb-2"
                type="date"
                value={da}
                onChange={(e) => setDa(e.target.value)}
            />

            <button
                className="btn btn-success mb-3"
                onClick={ab}
            >
                ADD TASK
            </button>

            <table className="table table-bordered table-striped">
                <thead className="table-dark">
                    <tr>
                        <th>Sr. No.</th>
                        <th>Task</th>
                        <th>Date</th>
                        <th>Action</th>
                    </tr>
                </thead>

                <tbody>
                    {
                        todo.map((a, index) => (
                            <tr key={index}>
                                <td>{index + 1}</td>
                                <td>{a.task}</td>
                                <td>{a.da}</td>
                                <td>
                                    <button
                                        className="btn btn-danger btn-sm"
                                        onClick={() => del(index)}
                                    >
                                        Delete
                                    </button>
                                </td>
                            </tr>
                        ))
                    }
                </tbody>
            </table>
        </div>
    );
};
```

Output :



TASK – 2:

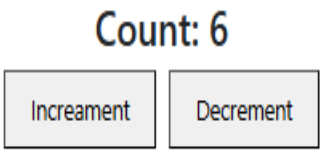
```jsx
import {useState} from "react"

export const Counter =()=>{
  const[count,setCount] = useState(0);

  return(
    <>
      <h3>Count: {count}</h3>
      <button onClick={() =>setCount(count +1) } style = {{padding: '10px 20px', fontSize: '16px', marginRight: '10px', cursor: 'pointer'}}>Increament</button>
      <button onClick={() =>setCount(count -1) } style = {{padding: '10px 20px', fontSize: '16px', cursor: 'pointer'}}> Decrement</button>

    </>
  )
}
```

Output :



Conclusion:

By the end of Day 18, I learned how to build dynamic React applications using `useState`, `useEffect`, array state management, and the `map()` method. Developing the Todo List and Counter applications strengthened my understanding of React Hooks and interactive user interfaces.